



EADN / API Runtime Signing with Eplan Cloud – A Guide for Developers

Valid as from version 2.3

Status: 07 / 2025

EPLAN GmbH & Co. KG
Technical information

Table of contents

Introduction	3
Notes for the Reader	3
Concept of EADN / API Runtime signing.....	5
Requirements	5
How you proceed.....	8
Modify the AssemblyInfo.....	8
Delay sign the Assemblies.....	8
Upload files manually.....	9
Check status of uploaded files	16
Receive the signed Assemblies manually	18
Delete uploaded files manually.....	20
Fully automated signing process	22
Special Cases.....	29
How to sign automatically generated serialization dlls	29
Signing of your own COM interop dlls.....	29

Introduction

Dear customer,

This document applies to the following scenarios:

Scenario 1

You already have a licensed Eplan product that is suitable for extension via API and the Eplan API extension license as well as the corresponding know-how about the API.

Scenario 2

You purchased the EADN Partner.

This document describes how you or your developers proceed, to get properly signed API modules.

Notes for the Reader

Before you begin reading, please note the following symbols and styles used in this document:



Warning:

Text preceded by this symbol contains a warning; you should be absolutely sure to read this warning before proceeding!!



Note:

Text preceded by this symbol contains extra notes.



Tip:

Useful tips to facilitate your interaction with the program are presented after this image.



Example:

Examples are highlighted by this symbol.

- User interface elements are marked in **bold** so they can immediately be located in the text.
- *Italic* text provides particularly important information that you should definitely pay attention to.
- Code examples, directory names and direct entries (among other things) are displayed in a `non-proportional` font.
- Function keys, keyboard keys, and buttons within the program are shown in square brackets (e.g., **[F1]** for the "F1" function key).
- To improve the flow of the text, we often use "menu paths" in this document (for example, **Help > Contents**). In order to find a particular program function, the menus and options shown in a menu path must be selected in the sequence shown. For example, the menu path mentioned above calls up the Eplan help system.
- The operating information provides step-by-step instructions for reaching specific targets, and in many places, after the right-arrow symbol $\Rightarrow$ you'll also find the program's reaction to your action.
- In combination with settings or fields (e.g., check boxes) which can only be switched on or off, in this document we often use the term "activate" (after which the setting is active ☒) and "deactivate" (after which the setting is inactive ☐)

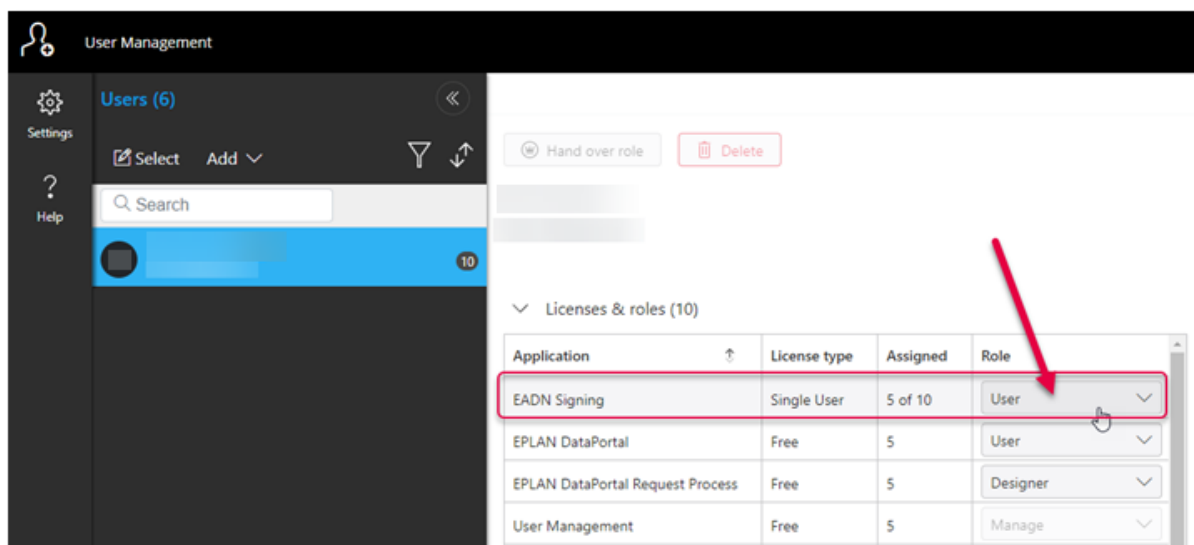
Concept of EADN / API Runtime signing

EADN / API Runtime signing uses a concept of combining standard .Net strong naming with additionally including an Eplan license option to the software. To achieve this combination, please follow the instructions in this document.

Requirements

After you have concluded your EADN contract or purchased an API Runtime license and you created a new software interface with Eplan, the main administrator of your Eplan cloud organization will get informed about a new entitlement for using the cloud-based EADN Signing service. Additionally, you will receive a file containing the public part of a standard signature key, normally used for strong-naming a .Net assembly. We created this key especially for your software.

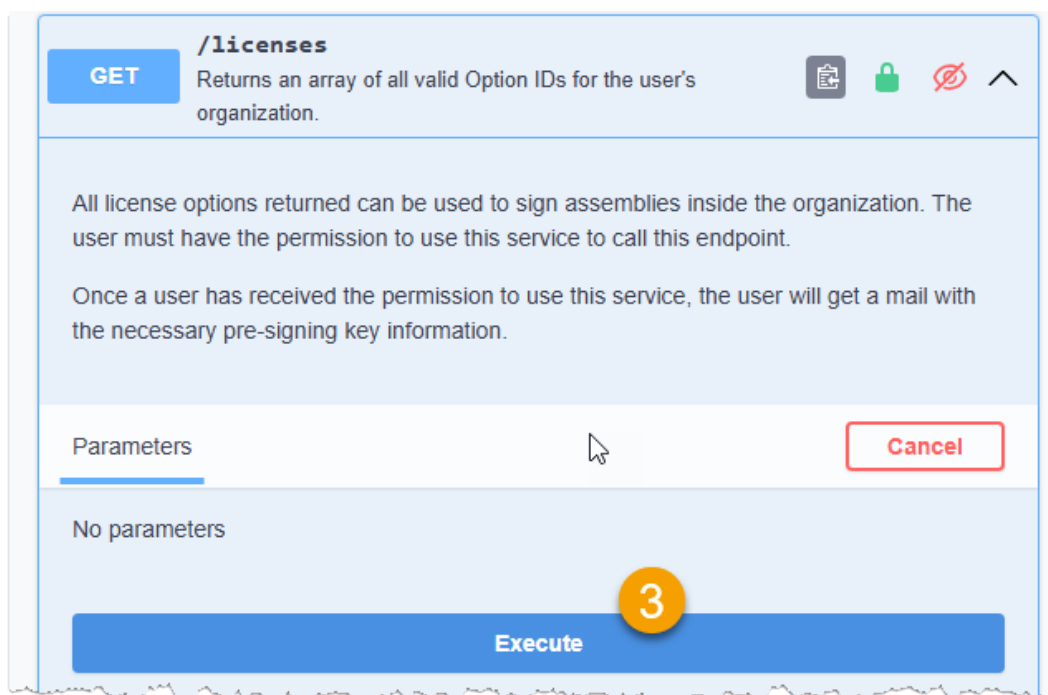
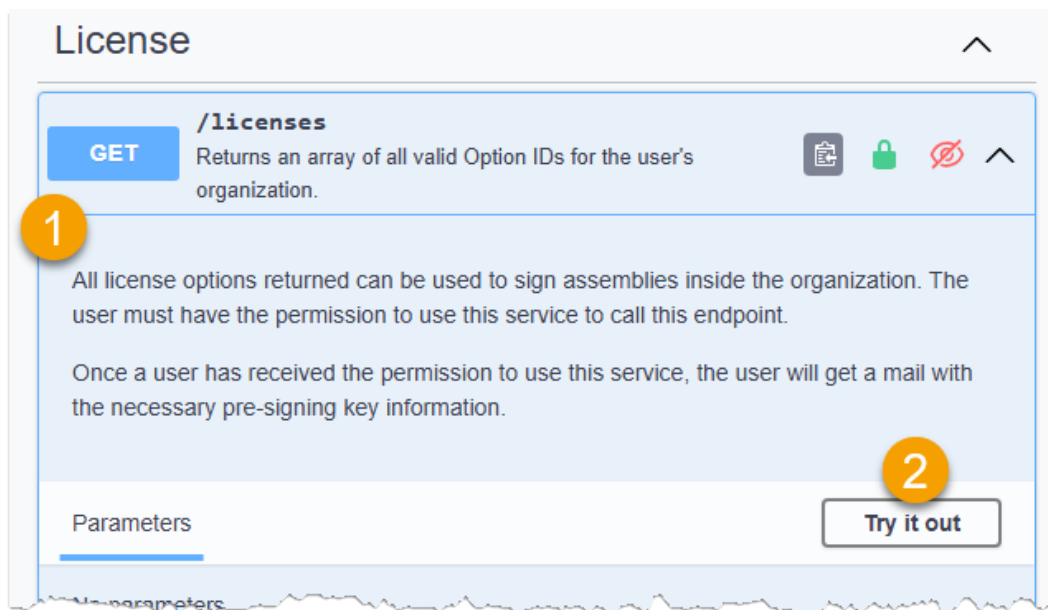
For using the Eplan cloud-based signing service, you have to be member of the regarding organization and got assigned the role **User** to the application **EADN Signing**:



Check available licenses inside active organization

Public pre-signing keyfiles (e.g. `1234_public.snk`, refer to [Delay sign the Assemblies](#)) which are used inside a Visual Studio project, must match to the used Eplan Cloud organization.

To verify, which pre-signing keyfiles can get used inside the active Eplan Cloud organization, authorize (refer to [Authorize manual usage of EADN-Signing](#)) and use the endpoint **GET /licenses** and check the result.





1. Click on endpoint **GET /licenses**
→ Section gets expanded and show more details
2. Click on **Try out**
3. Click on **Execute**
4. Check Response of server in section **Code**

Above example shows property **optionId** with value **1234**, which means assemblies with the used pre-signing keyfile `1234_public.snk` would be allowed to get signed inside the active Eplan Cloud organization.

Example of response, if no pre-signing key is assigned to current organization → organization not allowed to sign assemblies.



Note:

In case no valid license (**optionId**) is shown, please use [Eplan Global Support Portal](#) for getting help.

How you proceed

Take the following steps to get your application EADN / API Runtime - signed:

Modify the AssemblyInfo

In your software projects, you need to add an additional attribute to your AssemblyInfo files of all the assemblies, which are referencing Eplan API Assemblies. The EplanSignedAssemblyAttribute is implemented in the Eplan.EplApi.Starter.dll, which you always have to reference in your API application. The following example shows how to use the attribute in your AssemblyInfo file:

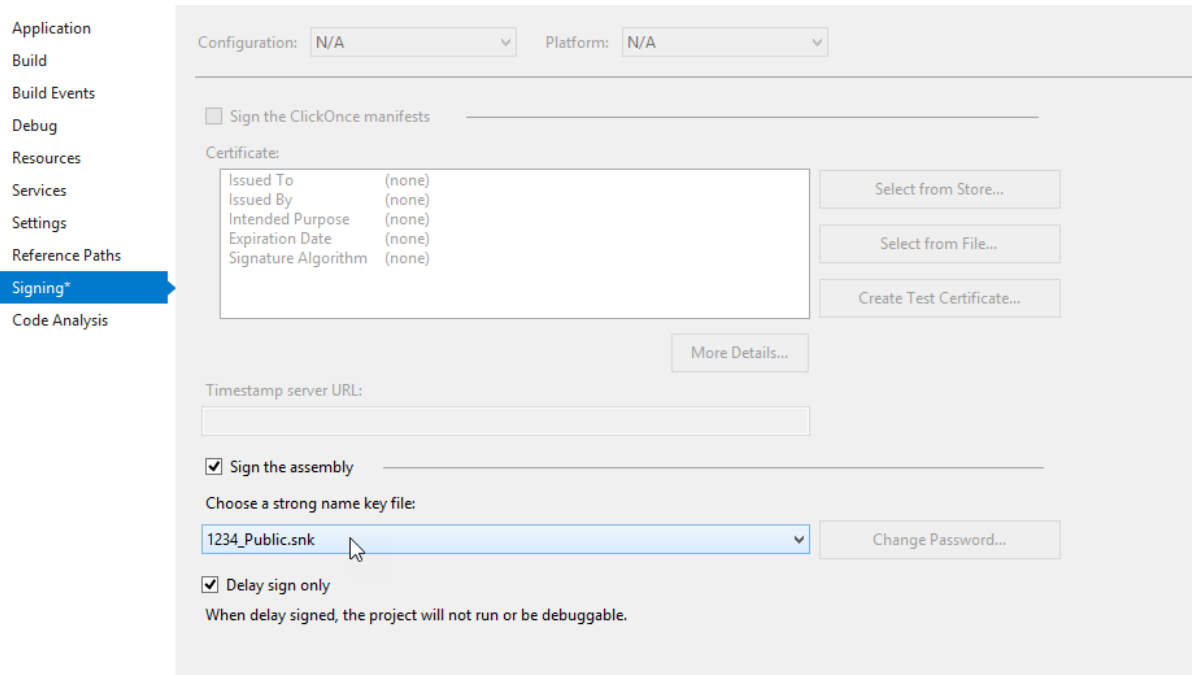


Example:

```
using System.Reflection;
using System.Runtime.CompilerServices;
using System.Runtime.InteropServices;
using Eplan.EplApi.Starter;
//...
[assembly: EplanSignedAssemblyAttribute(true)]
```

Delay sign the Assemblies

The easiest way for delay signing your assemblies (dll or exe) is entering the public key file in the signing properties of your software projects in Visual Studio. Check "Sign the assembly" and activate ☒ the "Delay sign only" flag. See the following image:



The screenshot shows the 'Signing*' configuration window in the Eplan Cloud Developer Portal. On the left is a sidebar with navigation links: Application, Build, Build Events, Debug, Resources, Services, Settings, Reference Paths, **Signing***, and Code Analysis. The main panel has a 'Configuration: N/A' and 'Platform: N/A' dropdowns. Below these, there's a checkbox for 'Sign the ClickOnce manifests'. A 'Certificate:' section contains a table with fields: Issued To, Issued By, Intended Purpose, Expiration Date, and Signature Algorithm, all currently set to '(none)'. To the right of this table are buttons: 'Select from Store...', 'Select from File...', and 'Create Test Certificate...'. Below the table is a 'More Details...' button. Further down is a 'Timestamp server URL:' text box. Below that is a checked checkbox for 'Sign the assembly'. Underneath is a label 'Choose a strong name key file:' followed by a dropdown menu showing '1234_Public.snk' and a 'Change Password...' button. At the bottom, there's a checked checkbox for 'Delay sign only' with a note: 'When delay signed, the project will not run or be debuggable.'

The delay signing is done, when building the software project and with it creating the assembly.

Alternatively, you can use Microsoft's Assembly Linker "Al.exe" to manually delay sign assemblies. Please refer to respective MSDN documentation.

Upload files manually

Log in to the Eplan Cloud Developer Portal <https://developer.eplan.com>



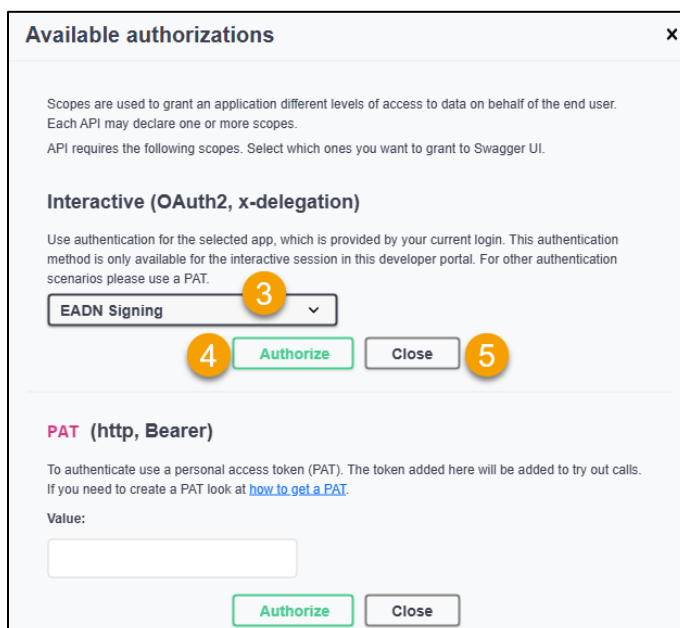
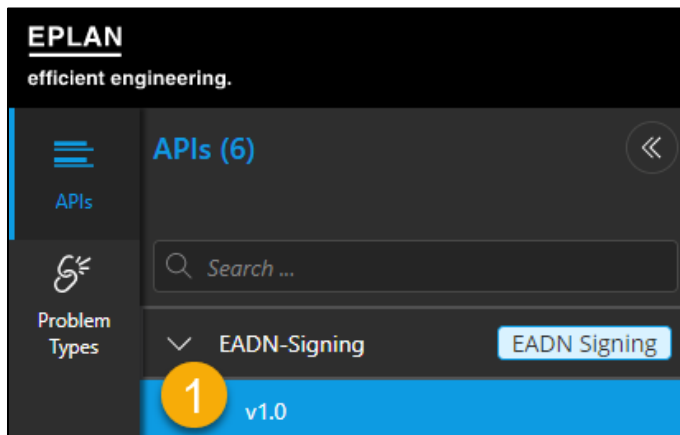
Notice:

Eplan Cloud account required to access the Developer Portal.



Note:

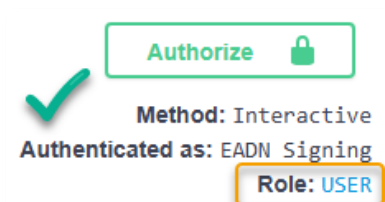
Manual signing via the Developer Portal is intended only for testing. The main purpose of the service is **automatic signing** (refer directly to [Fully automated signing process](#)).



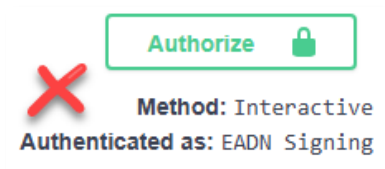
Authorize manual usage of EADN-Signing

1. Select **EADN-Signing** in available list of APIs
2. Click on **Authorize**
 - The authorization dialogue opens
3. Click the drop-down list and select **EADN Signing**
4. Click on **Authorize**
 - The API is allowed for manual usage via **Try it out** feature
5. Click on **Close**
 - The authorization dialogue is closed
 - Activated authorization is shown via closed lock symbol 

After successful authorization, the entry **USER** is displayed as active role:



Example when authorization was not successful:



Tip:

In case no valid role **USER** is shown, please refer to [Requirements](#) or contact the administrator of your Eplan Cloud organization. Same method can get used to verify, if a PAT is (*still*) valid. Just use authorization method **PAT** instead of **Interactive (OAuth2, x-delegation)**.



Notice:

When a different API is selected in the list of available APIs, an active authorization is discarded. Repeat the above steps to perform an authorization again.

Upload pre-signed assemblies / executables



1 **POST** `/assemblies` Upload one or multiple assemblies as one signing job

Upload one or multiple assemblies as multipart form-data to be signed. Additionally the upload can be extended by a comment, which is available in the list response. You can use this parameter to give some more details to identity an upload later. After the upload has succeed the files are going to be processed. This might take some time, therefore the status URI can be queried to check whether the processing has finished. If the processing has finished the download endpoint will provide the signed files in a zip archive.

2 Parameters [Try it out](#)

No parameters

Request body multipart/form-data

POST

/assemblies

Upload one or multiple assemblies as one signing job

No parameters

Request body

multipart/form-data

The body can contain multiple files. One file per section can be uploaded. The filename and the body of the file must be given. The maximum upload size is 40MB. The *files* parameter can either accept directly multiple files as separate sections inside the multipart content or the parameter needs to be named *files[0]* with an increasing number, starting from zero. The files are allowed to have a content type of *application/x-msdownload* (.dll) or *application/octet-stream* (.exe)

files * required

This field allows to upload a list of assemblies.

array

4

Choose File

EADN_SigningServi... Testproject_VALID.dll

-

3

Add file item

comment

string

Each upload can be extended with an additional comment.

5

My upload comment

☐

Send

empty value

6

Execute

Execute

Clear

Responses

Curl

```
curl -X 'POST' \
  'https://api.eplan.com/eadn-signing/v1.0/assemblies' \
  -H 'accept: application/json' \
  -H 'Authorization: Bearer eyJhbGciOiJSUzI1NiIsImtpZCI6IjE5Q0FFOURGRURDMUU2MjA3MzBF' \
  -H 'Content-Type: multipart/form-data' \
  -F 'files=@EADN_SigningService_Testproject_VALID.dll;type=application/x-msdownload' \
  -F 'comment=My upload comment'
```

Request URL

```
https://api.eplan.com/eadn-signing/v1.0/assemblies
```

Server response

Code	Details
202	Response body <pre>{ "id": "64175447-6ee5-4f05-86df-e0e6e0aae7a6", "links": { "status": "https://api.eplan.com/eadn-signing/v1.0/assemblies/64175447-6ee5-4f05-86df-e0e6e0aae7a6/status", "download": "https://api.eplan.com/eadn-signing/v1.0/assemblies/64175447-6ee5-4f05-86df-e0e6e0aae7a6", "delete": "https://api.eplan.com/eadn-signing/v1.0/assemblies/64175447-6ee5-4f05-86df-e0e6e0aae7a6" } }</pre>

1. Click on endpoint **POST /assemblies**
 - Section gets expanded and show more details
2. Click on **Try out**
 - Enables value entry in the **Request Body** section
3. Click on **Add file item**
 - Adds a new **Choose file** row
4. Click on **Choose file**
 - File selection dialogue opens
 - Select pre-signed assembly
 - Confirm with **Open** to add local file for upload
 - Repeat steps 3 and 4 for each file which needs to get signed
5. (Optional) Add personal **Comment** for full upload-job
 - The authorization dialogue is closed
6. Click on **Execute**
 - All selected files will get uploaded and tried to get signed
7. Check Response of server in column **Code**
8. Select and copy value of **id** in **Response body**, for later usage of all files

Check status of uploaded files

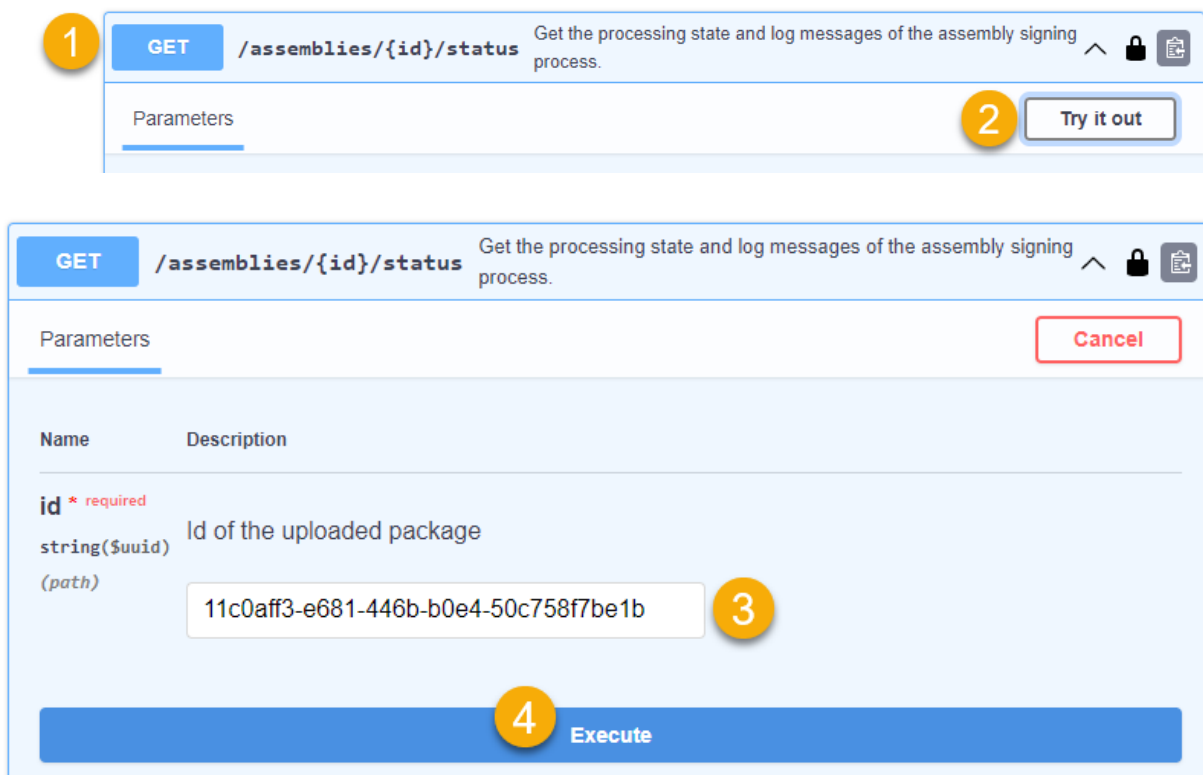
Log in to the Eplan Cloud Developer Portal <https://developer.eplan.com>

If required, perform [Authorize manual usage of EADN-Signing](#)

Make sure you have the **id** of the desired upload process handy
(see also *Select and copy value of id in Response body*)

Notice:

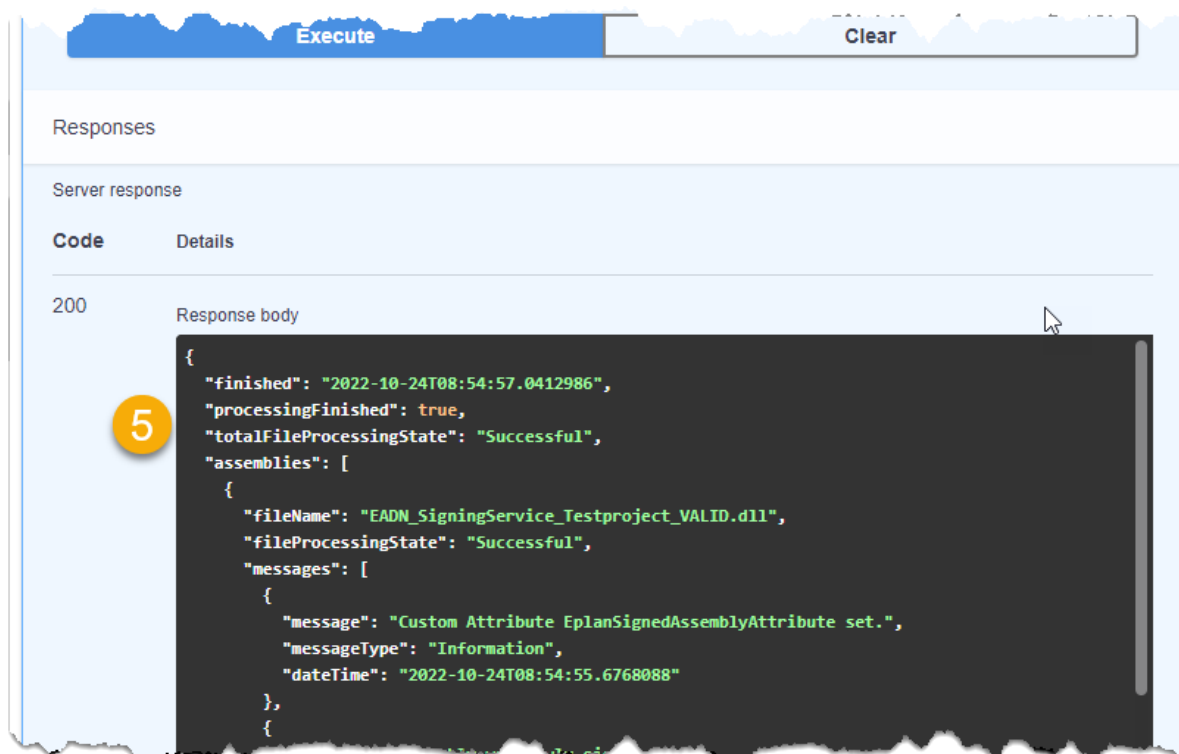
Use the endpoint **GET /assemblies** to return a list of all existing uploaded packages inside your organization and determine the required id.



The screenshot shows the API configuration interface for the endpoint `GET /assemblies/{id}/status`. The description is "Get the processing state and log messages of the assembly signing process." The interface includes a "Parameters" section with a table of parameters:

Name	Description
id * required	
string(\$uuid)	Id of the uploaded package
(path)	

The value `11c0aff3-e681-446b-b0e4-50c758f7be1b` is entered in the input field for the `id` parameter. The interface also features a "Try it out" button (labeled 2) and an "Execute" button (labeled 4). A "Cancel" button is also present in the top right of the parameters section.



1. Click on endpoint **GET /assemblies/{id}/status**
→ Section gets expanded and show more details
2. Click on **Try it out**
→ Enables value entry in the **Parameters** section
3. Insert previous copied id into parameter **Id of the uploaded package**
4. Click on **Execute**
→ Details about given signing job are getting returned
5. Check response of server in section **Responses** and get details about results of signing process

Receive the signed Assemblies manually

Log in to the Eplan Cloud Developer Portal <https://developer.eplan.com>

If required, perform *Authorize manual usage of EADN-Signing*

Make sure you have the **id** of the desired upload process handy
(see also *Select and copy value of id in Response body*)

Notice:

Use the endpoint **GET /assemblies** to return a list of all existing uploaded packages inside your organization and determine the required id.

1

GET

/assemblies/{id}

Download a zip archive file of the signed assemblies.



The download contains all signed and not signed files. To find out whether the signing was successful or not for a file inside of the archive check the status endpoint.

Parameters

2

Try it out

Parameters

Cancel

Name	Description
id * required	
string(\$uuid)	Id of the uploaded package
(path)	11c0aff3-e681-446b-b0e4-50c758f7be1b 3

4

Execute

Clear



1. Click on endpoint **GET /assemblies/{id}**
→ Section gets expanded and show more details
2. Click on **Try it out**
→ Enables value entry in the **Parameters** section
3. Insert previous copied id into parameter **Id of the uploaded package**
4. Click on **Execute**
→ Download request is getting executed
5. Click on **Download file** in section **Responses**
→ **Save file** selection dialogue opens
→ Confirm with **Save** to download the file

Delete uploaded files manually

Log in to the Eplan Cloud Developer Portal <https://developer.eplan.com>

If required, perform *Authorize manual usage of EADN-Signing*

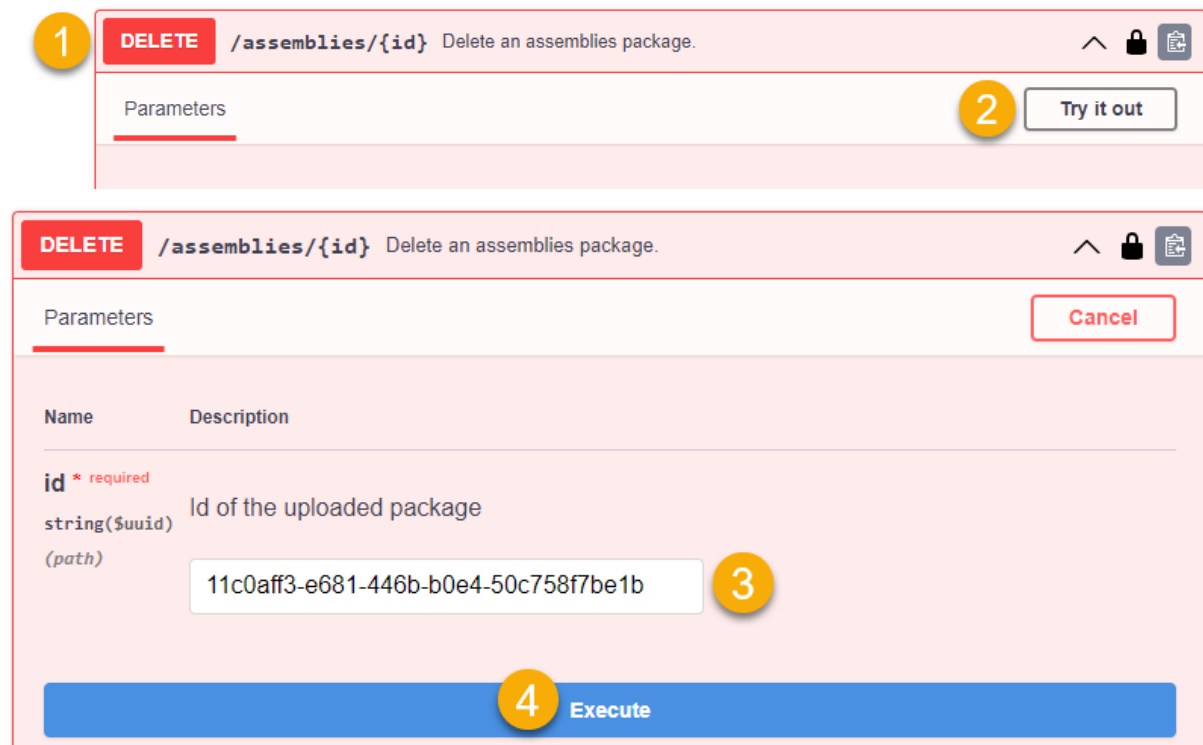
(see also [Limitations](#))

Make sure you have the **id** of the desired upload process handy

(see also *Select and copy value of id in **Response body***)

Notice:

Use the endpoint **GET /assemblies** to return a list of all existing uploaded packages inside your organization and determine the required id.



The screenshot shows the API interface for deleting an assembly package. It consists of two parts: a top summary card and a main parameters form.

Top Summary Card:

- Method: **DELETE**
- Endpoint: `/assemblies/{id}`
- Description: Delete an assemblies package.
- Parameters section: A "Try it out" button is visible.

Main Parameters Form:

- Method: **DELETE**
- Endpoint: `/assemblies/{id}`
- Description: Delete an assemblies package.
- Parameters section: A "Cancel" button is visible.
- Table with 2 columns: Name, Description
- Parameter **id** (required):
 - Type: `string($uuid)`
 - Description: Id of the uploaded package
 - Value: `11c0aff3-e681-446b-b0e4-50c758f7be1b`
- Execute button: A large blue button labeled "Execute".



1. Click on endpoint **DELETE /assemblies/{id}**
→ Section gets expanded and show more details
2. Click on **Try it out**
→ Enables value entry in the **Parameters** section
3. Insert previous copied id into parameter **Id of the uploaded package**
4. Click on **Execute**
→ Delete request is getting executed
5. Check result request in column **Code** in section **Responses**

Fully automated signing process

Preparations / prerequisites

To take advantage of fully automated signing of assemblies during the build process of Visual Studio, you must create a personal access token (further called **PAT**) for the application **EADN Signing** inside the profile editor of your Eplan Cloud organization.

See Eplan Cloud help:

- [Open My Settings](#)
- [Add personal access token \(PAT\)](#)
- [Roles and permissions](#)



Notice:

Without assigned role **User** to **EADN Service**, no **PAT** creation is available for the user.

Removing already assigned role **User** from **EADN Signing**, makes existing PAT invalid.

No prolonging of already created **PAT** (*new one needed after expiration*)
E-Mail notification is automatically sent from Eplan Cloud.

Download the provided PowerShell script from the [Developer Portal EADN Signing](#) for using it in Post-build event of Visual Studio.


EADN Signing - v1.0

Tags: [EADN Signing](#)

API for signing EPLAN assemblies. This API allows to sign assemblies like in the following guide explained: [EADN signing guide - EPLAN Cloud.pdf](#). Further information can also be found in API help of EPLAN platform: [API Miscellaneous - Signing EPLAN assemblies](#)

An example powershell script for automatic signing of EPLAN assemblies via post-build events can be downloaded via EPLAN Download Portal:

- Worldwide (except China): [Download EADN Signing PowerShell Script](#)
- China: [Download EADN Signing PowerShell Script](#)

Note: For accessing this download, You have to be already an EPLAN customer, have a valid dongle ID (serial number), customer number and an assigned EPLAN API Extension license module.

An example for calling the script including available parameters can be found inside the script itself:

```

1 param (
2     ....[array]$assemblies,
3     ....[string]$accessToken,
4     ....[string]$baseUrl = "https://api.eplan.com/eادن-signing/v1.0",
5     ....[string]$comment,
6     ....[string]$destinationPath,
7     ....[switch]$deleteAfterwards
8 )
9
10 # Example command line for running this script:
11 #
12 # powershell -ExecutionPolicy Bypass -file "<YourFolderName>\PostBuildScript.ps1"
  
```



Notice:

Depending on your IT guidelines, calling PowerShell scripts without bypassing executions policy may return an error. The active policy can get checked via running command `Get-ExecutionPolicy -List`



Tip:

See also [Microsoft documentation](#).

Parameter description

Parameter name	Mandatory	Description
<code>baseUrl</code>	No	Only for signing in Chinese environment; use parameter value*: <code>https://api.eplan.com.cn/eadn-signing/v1.0</code>
<code>comment</code>	No	Comment for complete upload job
<code>accessToken</code>	Yes	PAT which was created in User profile / MySettings
<code>rootDirectory</code>	No	Local root folder, in which all required assemblies / executables are located. Parameter only needed, if subdirectories are required in result-package, for instance language depending satellite assemblies or such NOTE: No closing backslash ' \ ' allowed
<code>assemblies</code>	Yes	Filename(s) of assemblies / executables which have to get signed Use absolute path for each filename, when parameter <code>rootDirectory</code> is not used, otherwise up to three levels of sub-folders are supported and will be created in the result-package
<code>destinationPath</code>	Yes	Local target folder for downloading result-package (via <i>PowerShell script target directory is tried to get created if missing</i>) NOTE: No closing backslash ' \ ' allowed
<code>deleteAfterwards</code>	No	Delete upload-job after tried signing automatically (Note: storage quota limitations)

*current `baseUrl` can be viewed at any time in the [Developer Portal](#)

Example:

Command call for signing a single file

```
powershell -ExecutionPolicy Bypass -file "<YourFolder-Name>\PostBuildScript.ps1" -comment "This is a test comment from $(USERNAME)" -accessToken "<YourPATforEADNSigningService>" -assemblies "<SourceFolderWithPreSignedFiles>\$(TargetFileName)" -destinationPath "<TargetFolder>" -deleteAfterwards
```

Example:

Command call for signing multiple files (no subfolders in ZIP result)

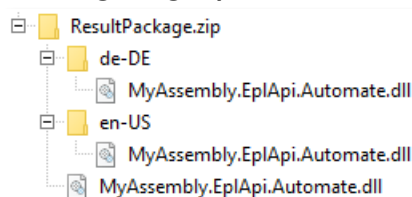
```
powershell -ExecutionPolicy Bypass -file "<YourFolder-Name>\PostBuildScript.ps1" -comment "This is a test comment from $(USERNAME)" -accessToken "<YourPATforEADNSigningService>" -assemblies "<SourceFolder1>\<YourFile1.dll>,<SourceFolder2>\<YourFile2.dll>" -destinationPath "<TargetFolder>" -deleteAfterwards
```

Example:

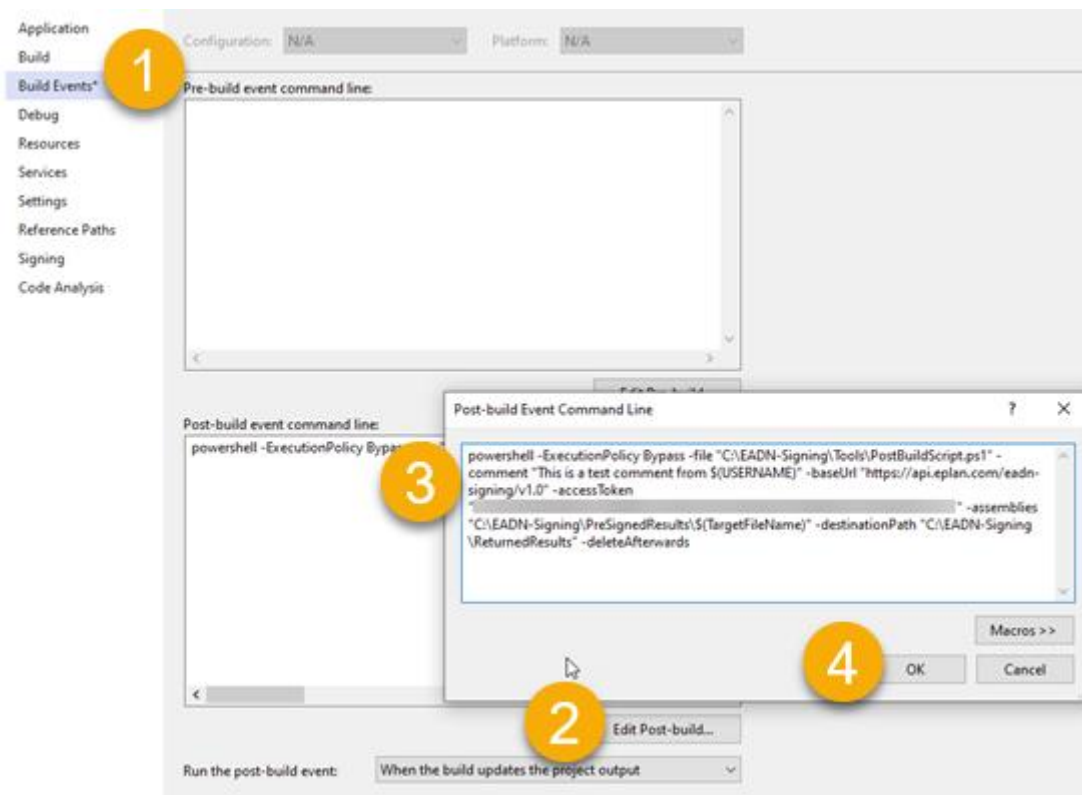
Command call for signing multiple files (subfolders in ZIP result)

```
powershell -ExecutionPolicy Bypass -file "<YourFolder-Name>\PostBuildScript.ps1" -comment "This is a test comment from $(USERNAME)" -accessToken "<YourPATforEADNSigningService>" -rootDirectory "<RootSourceFolder>" -assemblies "<RelativeFolder1>\<YourFile1.dll>,<RelativeFolder2>\<YourFile2.dll>" -destinationPath "<TargetFolder>" -deleteAfterwards
```

This allows, for example, files with the same name to be processed in one signing operation.

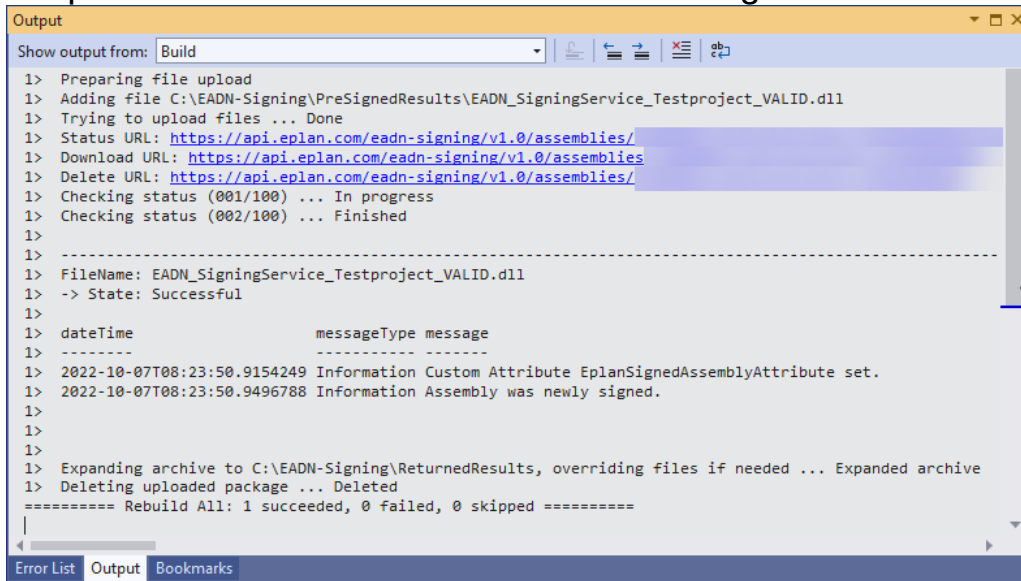


Adopt command line for PostBuild-event in Visual Studio



1. Click on **Build events**
→ Pre- and Post-build details of Visual Studio are shown
2. Click on **Edit Post-build...**
→ Post-build Event Command Line dialog opens
3. Paste required command line call
4. Click on **OK**
→ Post-build Command Line dialog is closed

Output console will show details after building:



```

Output
Show output from: Build
1> Preparing file upload
1> Adding file C:\EADN-Signing\PreSignedResults\EADN_SigningService_Testproject_VALID.d11
1> Trying to upload files ... Done
1> Status URL: https://api.eplan.com/eadn-signing/v1.0/assemblies/
1> Download URL: https://api.eplan.com/eadn-signing/v1.0/assemblies/
1> Delete URL: https://api.eplan.com/eadn-signing/v1.0/assemblies/
1> Checking status (001/100) ... In progress
1> Checking status (002/100) ... Finished
1>
1> -----
1> FileName: EADN_SigningService_Testproject_VALID.d11
1> -> State: Successful
1>
1>
1> dateTime          messageType message
1> -----
1> 2022-10-07T08:23:50.9154249 Information Custom Attribute EplanSignedAssemblyAttribute set.
1> 2022-10-07T08:23:50.9496788 Information Assembly was newly signed.
1>
1>
1> Expanding archive to C:\EADN-Signing\ReturnedResults, overriding files if needed ... Expanded archive
1> Deleting uploaded package ... Deleted
===== Rebuild All: 1 succeeded, 0 failed, 0 skipped =====
  
```

All files (no matter if signing process, was successful or not) will get extracted to given folder in parameter `destinationPath`

Limitations

- The filenames in one upload have to be unique. Adding the same filename multiple times (*on same folder level*) is not allowed, because it can not be reflected in the ZIP-file for download.
- Each upload job can have a max. total file size of ~40 MB
- There is an upload limit of max. 9999 kept upload jobs for each organization.
- No automatic “cleanup” is done in organization storage.
- If upload limit is reached, older uploaded files have to get deleted before new uploads are possible.

Tip:

It is recommended to delete uploaded files directly after signing (*no matter if signing was successful or not*), to avoid reaching the upload limit at all.



Notice:

Further details can be also found in the Developer Portal:

[ProblemTypes of EADN signing](#)

Special Cases

How to sign automatically generated serialization dlls

If you use an automatically created serialization dll for your classes, you need to delay-sign them via the sgen.exe tool. This tool can be found in the SDK directory of your development environment.



Example:

```
"C:\Program Files\Microsoft Visual Studio
8\SDK\v2.0\bin\sgen.exe" /compiler:/delaysign+ /assem-
bly:"MyDllToBeSerialized.dll" /proxytypes
/reference:"Eplan.EplApi.AFu.dll" /reference:<...all fur-
ther references you need> /com-
piler:/keyfile:"D:\MyPublicKey.snk"
```

Signing of your own COM interop dlls

As you probably know, any strong-named .Net assembly can only reference / load other strong-named assemblies. In case your application registers COM dlls, the development environment normally automatically creates so-called interop dlls, which contain the .Net wrapping of the respective COM methods. Normally, these dlls are not signed. To create these assemblies in an already delay-signed way Microsoft provides the command line tool tlbimp.exe – also to be found in the SDK directory of your development environment. See the following example, how it is used:



Example:

```
"C:\Program Files\Microsoft Visual Studio
8\SDK\v2.0\Bin\tlbimp.exe" yourComInterface.dll /delaysign
/publickey:C:\YourKeyFilePublic.snk /out:Interop.yourCom-
Interface.dll
```



EPLAN GmbH & Co. KG
An der alten Ziegelei 2
40789 Monheim am Rhein · Germany
Telefon: +49 (0) 21 73 / 39 64-0
Telefax: +49 (0) 21 73 / 39 64-25
Info@eplan.de · www.eplan.de